



Security Audit

AIMA (FinTech)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	12
Audit Resources	12
Dependencies	13
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	35
Conclusion	36
Our Methodology	37
Disclaimers	39
About Hashlock	40

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The AIMA team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

AIMA is an AI-powered financial market intelligence platform designed to analyze fragmented data across crypto, equities, foreign exchange, and commodities. It uses a multi-agent architecture combining forecasting, risk, sentiment, market-structure, and explainability layers to generate structured, probabilistic market insights for users. The project is positioned as infrastructure for institutional-grade financial intelligence, with planned Web3 and token-based ecosystem components.

Project Name: AIMA

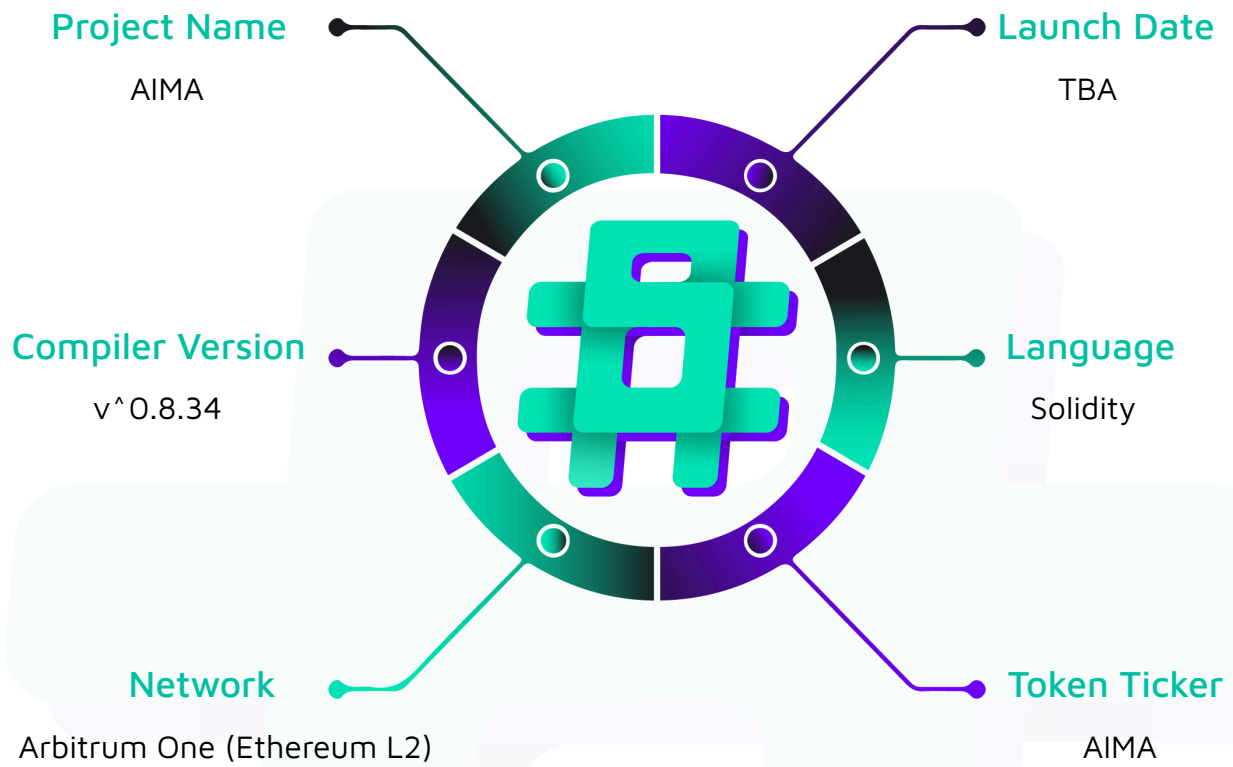
Project Type: FinTech

Compiler Version: ^0.8.34

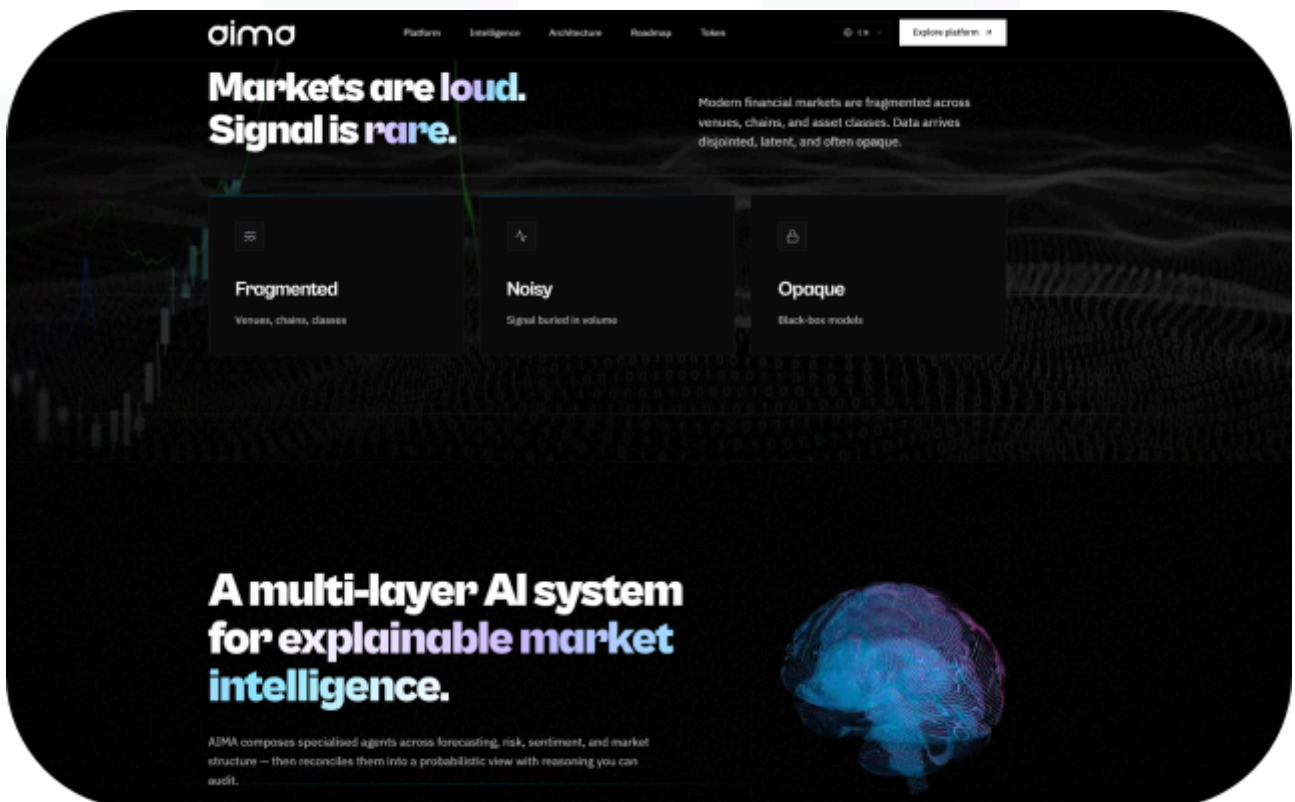
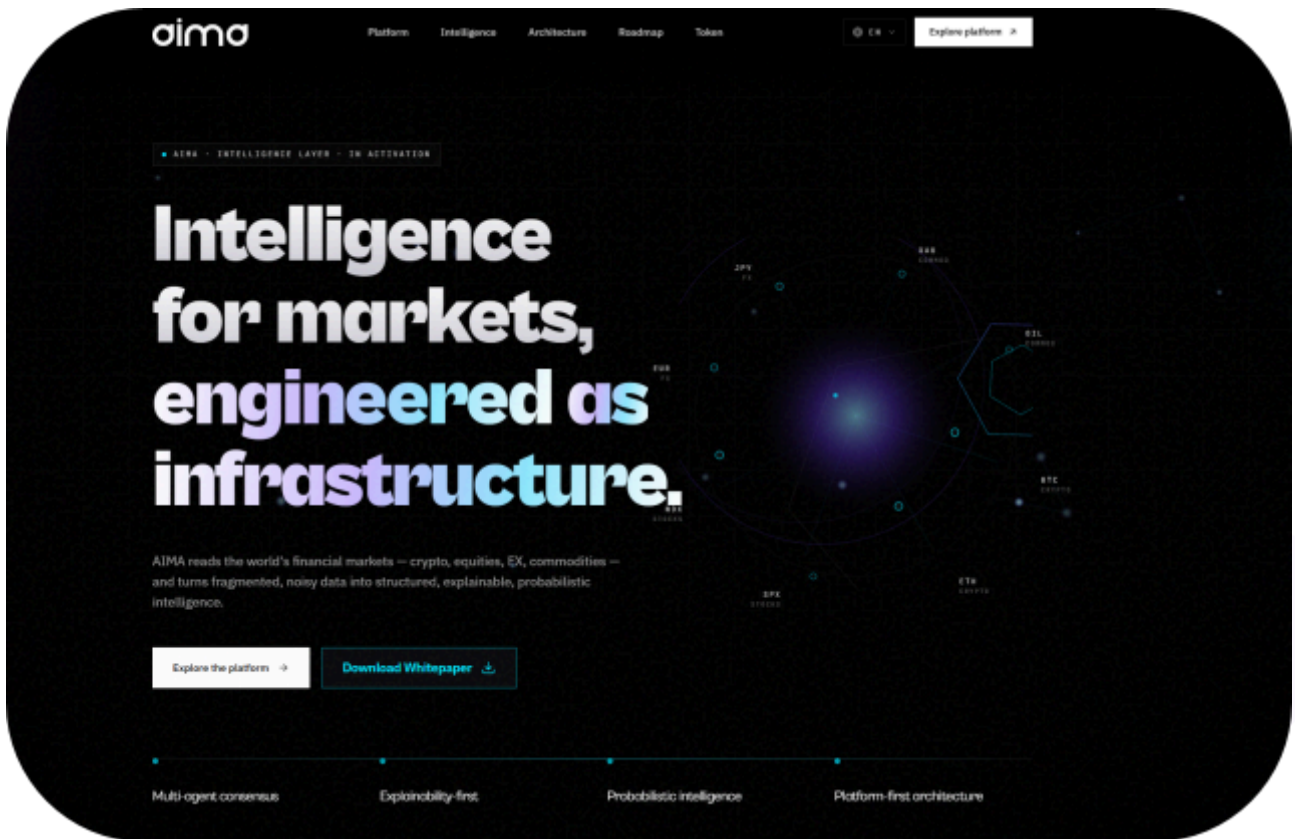
Website: <https://aima.market>

Logo:



Visualised Context:

Project Visuals



Audit Scope

We at Hashlock audited the solidity code within the AIMA project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	AIMA
Network	Arbitrum One (Ethereum L2)
Language	Solidity
Audit Date	August, 2026
Contract 1	AIMAToken.sol
Contract 2	AIMASale.sol
Contract 3	AIMAVesting.sol
Contract 4	IAIMAVesting.sol
Contract 5	AIMAConstants.sol
Contract 6	AIMADataTypes.sol
Contract 7	AIMAErrors.sol
Contract 8	AIMAEvents.sol
Audited GitHub Commit Hash	920a4b4e3b4b174427203b4def4cf4ffa1c4af14
Fix Review GitHub Commit Hash	e652dcd5dbd21f09b2bd3751950f8f3a813d88de

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.



Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

2 Medium severity vulnerabilities

7 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
AIMAToken.sol - Fixed-supply ERC-20 named AIMA, 18 decimals. - Mints the entire 25,000,000 AIMA supply to the Safe at construction. - No additional minting, ever. - Non-upgradeable. - Emergency pause capability restricted to the Safe.	Contract achieves this functionality.
AIMASale.sol - Manages the capped 3,750,000 AIMA Private Sale with one unresolved round at a time. - Accepts USDC or USDT payments, with both payment tokens using 6 decimals. - Uses Round 1 as the only tiered round: the first 1,000,000 AIMA is priced at EUR 0.20, the next 1,000,000 at EUR 0.25, and the remaining Round-1 allocation at EUR 0.30. - Establishes the common buyer Token Activation Date as Round 1 startTimestamp + 45 days. - Requires every subsequent round to end no later than TAD + 18 contract months. - Assigns purchases from subsequent fixed-price rounds to Tier-3 buyer vesting against the same common TAD - Protects purchases with minTokensOut, expectedRoundId - transfers purchased AIMA to AIMAVesting and	Contract achieves this functionality.

creates buyer grants instead of transferring tokens directly to buyers - Teats individual round closure and global Private Sale finalization as separate lifecycle events. Allows the Safe to finalize early and permits anyone to finalize after the allocation is sold out or the absolute longstop is reached, provided no round remains unresolved. - Tracks round-level and global sale totals, supports Safe withdrawal of payment proceeds, returns unsold AIMA after finalization, and provides Safe-controlled rescue flows for accidental assets	
AIMAVesting.sol - Holds team and private-sale vesting. - Receives AIMA from the sale and creates buyer grants per tier. Manages grant lifecycle: pending, active, cancelled. - Team beneficiary recovery with timelock and contest flow. - Protected vesting reserves — rescue may never break solvency. - Independent pause control for claims.	Contract achieves this functionality with the deviation
libraries/AIMAConstants.sol - defines the system-wide constants used by all three contracts	Contract achieves this functionality.
libraries/AIMADataTypes.sol - Defines the enums and structs used across the system: VestingType, GrantStatus, RecoveryStatus, SaleRoundStatus, Grant,	Contract achieves this functionality.

RecoveryRequest, SaleRound.	
libraries/AIMAErrors.sol - Central custom-error definitions for revert reasons across all contracts.	Contract achieves this functionality.
libraries/AIMAEvents.sol - Central event definitions for all state transitions across the system.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the AIMA project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the AIMA project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] AIMASale#buyWithUSDC - Spend-denominated purchases with no minimum-output protection let a front-run at a tier boundary silently reprice the victim up to 33.33% worse

Description

The purchase functions take a payment amount and nothing else. A transaction prepared under one sale state executes under a different sale state without reverting: there is no minimum token output, expected round, expected rate, or deadline. A competing buyer can front-run a large purchase and push the victim across a global tier boundary. The victim's transaction still succeeds but buys materially fewer tokens for the same spend. A transaction prepared during one round can also execute unmodified in a later round with a different exchange rate.

Vulnerability Details

The public purchase functions bind only the spend amount:

```
function buyWithUSDC(uint256 paymentAmount) external nonReentrant whenNotPaused {
    _buy(address(usdc), paymentAmount);
}

function buyWithUSDT(uint256 paymentAmount) external nonReentrant whenNotPaused {
    _buy(address(usdt), paymentAmount);
}
```

The internal path obtains a fresh quote from current state without checking a buyer-provided output floor or round identifier:

```
function _buy(address paymentToken, uint256 paymentAmount) internal {
```

```
(uint256 totalTokens, uint256 tier1Tokens, uint256 tier2Tokens, uint256
tier3Tokens, uint256 paymentUsed) =
    previewPurchase(paymentToken, paymentAmount);
```

The quote reads the live global tier cursor:

```
uint256 paymentRemaining = paymentAmount;
uint256 tokensRemaining = availableTokens;
uint256 sold = totalSold;
```

Prices step from EUR 0.20 to EUR 0.25 to EUR 0.30 as cumulative sales pass 1 million and 2 million AIMA. Nothing compares the live output with the amount the buyer expected when submitting the transaction.

Proof of Concept

The executable tests are:

- test_PoC_Medium1_TierBoundarySilentRepricing
- test_PoC_Medium1_StaleTxExecutesInNextRoundAtNewRate

Focused tier-boundary sequence:

```
function test_PoC_Medium1_TierBoundarySilentRepricing() public {
    _openRound();

    (uint256 quotedTokens, , , ) = sale.previewPurchase(address(usdc), 216_000e6);
    assertEq(quotedTokens, 1_000_000e18);
    assertEq(sale.totalSold(), 0);

    uint256 frontRunPayment = 1_000_000e18 * TIER1_RATE / 1e24;
    vm.prank(attacker);
    sale.buyWithUSDC(frontRunPayment);
    assertEq(sale.totalSold(), AIMAConstants.TIER1_MAX);

    vm.prank(victim);
    sale.buyWithUSDC(216_000e6);
```

```

uint256 received = sale.totalBought(victim);
assertEq(received, 800_000e18);
assertEq(received * 5, quotedTokens * 4);
}

```

Focused round-rollover sequence:

```

function test_PoC_Medium1_StaleTxExecutesInNextRoundAtNewRate() public {
    _openRound();
    (uint256 quoted,,, ) = sale.previewPurchase(address(usdc), 216_000e6);
    assertEq(quoted, 1_000_000e18);

    vm.warp(block.timestamp + 31 days);
    vm.startPrank(safe);
    sale.closeRound(sale.currentRoundId());
    sale.createRound(
        uint64(block.timestamp), uint64(block.timestamp + 30 days),
        AIMAConstants.PRIVATE_SALE_ALLOCATION, 1_300_000, 1_300_000
    );
    vm.stopPrank();

    vm.prank(victim);
    sale.buyWithUSDC(216_000e6);
    uint256 received = sale.totalBought(victim);
    assertLt(received, quoted * 87 / 100);
}

```

Run command:

```
forge test --match-contract HashlockPoC -vv
```

Observed output against the audited source:

```

[PASS] test_PoC_Medium1_StaleTxExecutesInNextRoundAtNewRate()
[PASS] test_PoC_Medium1_TierBoundarySilentRepricing()
[PASS] test_PoC_Medium2_GuardianFreezeLoop()
Suite result: ok. 3 passed; 0 failed; 0 skipped.

```

The live re-quote correctly preserves sale and round caps, and all payment, token-transfer, and grant writes remain atomic. This finding is not a cap bypass or partial-transfer issue. The loss arises because the transaction succeeds at a materially



worse live quote that the buyer cannot bound. At a rate change from 1.08 to 1.30 within tier 1, output falls from 1,000,000 to approximately 830,769.23 AIMA, a 16.92% reduction.

Impact

For the same payment, moving entirely from tier 1 to tier 2 gives 20% fewer tokens, and moving from tier 2 to tier 3 gives 16.67% fewer tokens. A transaction displaced from the tier-1 price to the tier-3 price can receive up to 33.33% fewer tokens. Stale transactions are also exposed to inter-round exchange-rate changes. Any competing buyer can trigger the tier-boundary variant when a suitable purchase is visible in the mempool.

Recommendation

Bind buyer intent at execution. Extend the purchase API with `minTokensOut`, `expectedRoundId`, and `deadline`, then revert if the deadline passed, the round changed, or the live output is below the user's minimum. An optional maximum-rate argument can provide additional explicit protection.

Status

Resolved

[M-02] AIMAVesting#requestRecovery - A recovery guardian can repeatedly re-arm a freeze of a fully vested team grant and cannot be evicted

Description

Claim blocking begins as soon as recovery is requested, before the proposed beneficiary confirms and before the Safe approves. A malicious or compromised guardian can freeze a team beneficiary's claims with one transaction and repeatedly restore the freeze after Safe cancellation. The beneficiary has no self-cancel, cancellation has no cooldown or terminality, and the recovery address cannot be rotated or removed while the beneficiary retains the grant.

Vulnerability Details

A mere request is treated as active and blocks both the view and state-changing claim path:

```
if (grant.vestingType == AIMADataTypes.VestingType.TEAM &&
    _recoveryIsActive(grantId)) {
    revert AIMAErrors.RecoveryInProgress();
}
```

```
return (status == AIMADataTypes.RecoveryStatus.REQUESTED || status ==
    AIMADataTypes.RecoveryStatus.CONFIRMED
        || status == AIMADataTypes.RecoveryStatus.APPROVED || status ==
    AIMADataTypes.RecoveryStatus.CONTESTED
        || status == AIMADataTypes.RecoveryStatus.OVERRIDDEN);
```

Cancellation explicitly permits immediate re-entry:

```
if (
    currentStatus != AIMADataTypes.RecoveryStatus.NONE && currentStatus !=
    AIMADataTypes.RecoveryStatus.EXECUTED
        && currentStatus != AIMADataTypes.RecoveryStatus.CANCELLED
) {
    revert AIMAErrors.RecoveryAlreadyInProgress();
}
```

The guardian designation is one-shot:



```

if (recoveryAddress[msg.sender] != address(0)) {
    revert AIMAErrors.RecoveryAddressAlreadySet();
}

```

```

recoveryAddress[msg.sender] = recoveryAddress_;

```

`cancelRecovery` changes only request status and timing; it does not clear the guardian:

```

request.status = AIMADatatypes.RecoveryStatus.CANCELLED;
request.executeAfter = 0;

```

Proof of Concept

Executable test: `test_PoC_Medium2_GuardianFreezeLoop`

```

function test_PoC_Medium2_GuardianFreezeLoop() public {
    vm.startPrank(safe);
    vesting.createTeamGrant(beneficiary, AIMAConstants.TEAM_GRANT_AMOUNT);
    vm.stopPrank();
    uint256 gid = vesting.teamGrantIdOf(beneficiary);
    vm.prank(beneficiary); vesting.acceptTeamGrant(gid);
    vm.prank(beneficiary); vesting.setRecoveryAddress(gid, guardian);
    vm.warp(uint256(vesting.tokenCreationTimestamp()) + 24 *
uint256(AIMAConstants.MONTH) + 1);

    assertEq(vesting.releasableAmount(gid), AIMAConstants.TEAM_GRANT_AMOUNT);

    vm.prank(guardian); vesting.requestRecovery(gid, newX);
    assertEq(vesting.releasableAmount(gid), 0);
    vm.prank(beneficiary);
    vm.expectRevert(AIMAErrors.RecoveryInProgress.selector);
    vesting.claim(gid);

    vm.prank(safe); vesting.cancelRecovery(gid);
    vm.prank(guardian); vesting.requestRecovery(gid, newX);
    assertEq(vesting.releasableAmount(gid), 0);
    vm.prank(beneficiary);
    vm.expectRevert(AIMAErrors.RecoveryInProgress.selector);
    vesting.claim(gid);
}

```

```

vm.prank(safe); vesting.cancelRecovery(gid);
vm.prank(guardian); vesting.requestRecovery(gid, newX);
assertTrue(vesting.getRecoveryRequest(gid).status ==
AIMADataTypes.RecoveryStatus.REQUESTED);
assertEq(vesting.releasableAmount(gid), 0);
}

```

Run command:

```
forge test --match-contract HashlockPoC -vv
```

Observed output against the audited source:

```

[PASS] test_PoC_Medium1_StaleTxExecutesInNextRoundAtNewRate()
[PASS] test_PoC_Medium1_TierBoundarySilentRepricing()
[PASS] test_PoC_Medium2_GuardianFreezeLoop()
Suite result: ok. 3 passed; 0 failed; 0 skipped.

```

Impact

A malicious or compromised guardian can repeatedly deny access to a fully vested 1,250,000 AIMA team grant, equal to 5% of total supply per grant. A shared guardian can exercise the same power over multiple beneficiaries that selected it. Every defensive cycle requires another Safe cancellation, while the guardian retains permission to immediately re-arm the request.

Recommendation

Do not block claims during an unconfirmed REQUESTED state, or add bounded expiry and beneficiary self-cancellation. Make a cancellation terminal or impose a meaningful cooldown before re-entry. Add a controlled path for the beneficiary or Safe to rotate or evict a compromised recovery address without transferring the grant.

Status

Resolved

QA

[Q-01] AIMASale#createRound - Exchange rates have no sanity bounds and no refresh path

Description

Round creation only checks that both exchange rates are nonzero. A rate entered at the wrong scale is accepted and fixed for the round. Nothing constrains the two payment legs to agree with each other.

Vulnerability Details

```
if (eurToUsdcRate6 == 0 || eurToUsdtRate6 == 0) {  
    revert AIMASale.InvalidExchangeRate();  
}
```

No function updates an open round's rates. The available correction is to pause, close, and recreate the round.

eurToUsdcRate6 and eurToUsdtRate6 are independent inputs with no parity constraint. If they diverge, buyers route through whichever stablecoin prices AIMA lower, and the protocol receives the worse of the two rates on every sale.

Rate entry is unassisted. `script/ConfigureAIMASystem.s.sol` is disabled, so step 5 of the documented runbook ("Create Private Sale rounds through `AIMASale.createRound(...)` only when approved") is executed as raw Safe calldata with no script, no computation aid, and no post-entry verification.

Impact

A trusted configuration error can materially misprice sales already executed or make ordinary purchases return zero tokens and revert. Divergent legs leak sale proceeds for as long as the round remains open. This is an administrative configuration risk, not a non-admin privilege bypass.

Recommendation

Enforce a documented minimum and maximum rate band, and reject rounds whose two legs differ by more than a defined tolerance. Consider a minimum delay before a newly created round starts, so a misentered rate can be cancelled before any purchase lands.

Status

Resolved



[Q-02] AIMAVesting#setSale - Deployment wiring is unvalidated on-chain and unverified off-chain

Description

The sale binding accepts any nonzero address exactly once without verifying that the address is the intended sale contract, and the repository provides no compensating deployment-time check that would surface a mistake before the first purchase attempt.

Vulnerability Details

```
function setSale(address sale_) external onlySafe {
    if (sale != address(0)) revert AIMAErrors.SaleAlreadyConfigured();
    if (sale_ == address(0)) revert AIMAErrors.ZeroAddress();

    sale = sale_;
    emit AIMAEvents.SaleConfigured(sale_);
}
```

The same codebase demonstrates the opposite pattern one contract over. AIMASale's constructor refuses to deploy against mismatched wiring:

```
if (AIMAToken(aima_).safe() != safe_) {
    revert AIMAErrors.SafeMismatch();
}
```

```
if (address(vestingContract.aima()) != aima_ || vestingContract.safe() != safe_) {
    revert AIMAErrors.VestingConfigurationMismatch();
}
```

The wiring risk is therefore asymmetric: the sale validates its counterparties, the vesting contract does not validate its sale.

No deployment-side control compensates. `script/ConfigureAIMASystem.s.sol` is disabled:

```
function run() external pure {
```

```
    revert("Disabled: configure production through Safe only");  
}
```

Its five documented steps are a doc comment executed by hand through the Safe, and `AIMAVesting.setSale(AIMASale)` is step 1 — the first manual action of the deployment. The four deploy scripts (`DeployAIMAToken`, `DeployAIMAVesting`, `DeployAIMASale`, `DeployMocks`) read addresses from environment variables, call the constructor, and assert nothing after construction. The repository contains no readiness check asserting that `vesting.sale()` is the deployed sale, that vesting holds the team reserve, or that the sale holds the private-sale allocation.

Impact

A trusted deployment mistake in runbook step 1 permanently disables buyer-grant creation and sale finalization for that vesting instance, and nothing in the repository would reveal the error until the first purchase reverts. Purchases fail atomically, so this is fail-closed and does not take buyer payment.

Recommendation

Validate that the candidate sale reports the matching AIMA token and Safe, or permit tightly controlled re-binding while no buyer grants or close timestamp exist. Separately, add a deployment-readiness script or test that asserts the full wiring and funding preconditions before the first round is created; the same check also covers the funding precondition described in Q-04.

Status

Resolved

[Q-03] AIMASale#finalizePrivateSale - Buyer vesting depends on a discretionary step that the deployment runbook omits

Description

Buyer vesting cannot begin until the sale calls the vesting contract's close-timestamp setter. Nothing forces finalization, the 390-day TAD longstop begins only after that timestamp exists, and the project's own documented post-deployment procedure never mentions the call.

Vulnerability Details

Affected file: AIMASale.sol

```
privateSaleFinalized = true;
privateSaleCloseTimestamp = closeTimestamp;

vesting.setPrivateSaleCloseTimestamp();
```

Affected file: AIMAVesting.sol

```
if (privateSaleCloseTimestamp == 0) {
    return 0;
}
```

While `privateSaleCloseTimestamp` is zero, `effectiveTAD()` returns zero and `_computeVested` returns zero for every buyer grant, regardless of elapsed time.

Affected file: script/ConfigureAIMASystem.s.sol

```
/// Required post-deployment actions include:
/// 1. AIMAVesting.setSale(AIMASale)
/// 2. Transfer 5,000,000 AIMA from Safe to AIMAVesting for Team allocation
/// 3. Transfer 3,750,000 AIMA from Safe to AIMASale for Private Sale allocation
/// 4. Create the approved Team grants through the Safe
/// 5. Create Private Sale rounds through AIMASale.createRound(...) only when
approved
```

The documented procedure ends at round creation. Neither `finalizePrivateSale()` nor `setTAD()` appears in it, or anywhere else under `script/`. The omission is not hypothetical operator error; it is the state of the written runbook.

Impact

If the trusted Safe follows the documented steps and stops there, every paid buyer grant remains permanently unclaimable and the sale's remaining AIMA stays unrescuable, because `returnUnsoldAIMA` is itself gated on finalization. No untrusted actor can force or exploit this state for theft, so the issue is classified as Q under the audit trust model.

Recommendation

Add `finalizePrivateSale()` and `setTAD()` to the documented post-deployment actions as explicit, scheduled steps. In the contracts, add a bounded fallback for setting the close timestamp, or automatically permit finalization once the allocation is sold and a defined deadline has passed.

Status

Resolved

[Q-04] AIMAVesting - Team reserve is keyed to the `TEAM_ALLOCATION` constant, blocking the first purchase and stranding any unallocated remainder

Description

Both the buyer-grant solvency check and the rescue solvency check compute the protected team reserve from the `TEAM_ALLOCATION` constant rather than from `totalTeamAllocated`. The reserve therefore reflects the maximum team allocation that could ever be granted, not the amount actually granted. This single choice produces one entry-side and one exit-side consequence.

Vulnerability Details

Affected file: `AIMAVesting.sol` entry side, `createBuyerGrant`:

```
uint256 protectedTeamReserve = AIMAConstants.TEAM_ALLOCATION - totalTeamReleased;

uint256 newBuyerOutstanding = newBuyerAllocated - totalBuyerReleased;

if (aima.balanceOf(address(this)) < protectedTeamReserve + newBuyerOutstanding) {
    revert AIMAConstants.InsufficientVestingBalance();
}
```

The incoming buyer amount appears in both the balance and the required reserve and cancels out, so the pre-existing vesting balance must already cover the entire team allocation before the very first buyer grant can be created.

Affected file: `AIMAVesting.sol` - exit side, `rescueToken`:

```
uint256 protectedTeamReserve = AIMAConstants.TEAM_ALLOCATION - totalTeamReleased;
uint256 buyerOutstanding = totalBuyerAllocated - totalBuyerReleased;
uint256 requiredBalance = protectedTeamReserve + buyerOutstanding;

if (amount > balance || balance - amount < requiredBalance) {
    revert AIMAConstants.RescueWouldBreakSolvency();
}
```

`totalTeamReleased` increases only when a team beneficiary claims, which requires a grant to exist. If fewer than four 1,250,000 AIMA team grants are ever created and

claimed, the ungranted remainder can never satisfy `balance - amount >= requiredBalance` for any nonzero amount. There is no burn path and no claim path for it.

Runbook step 4 reads "Create the approved Team grants through the Safe", which leaves the number of grants open. A deployment that funds the full 5,000,000 AIMA reserve but approves fewer than four grants is a documented, ordinary path — not a corner case.

Impact

Entry side: incorrect deployment funding disables every purchase until the Safe funds the missing reserve. The transaction reverts atomically, so buyers do not lose payment.

Exit side: any portion of the 5,000,000 AIMA team reserve that is never granted is permanently immobilised in the vesting contract. With two grants approved instead of four, 2,500,000 AIMA — 10% of total supply — becomes unrecoverable once buyer obligations are settled.

The documented funding order is correct as written: steps 2 and 3 fund vesting and the sale before step 5 creates any round. The gap is that nothing enforces or verifies that order, and nothing releases the reserve once the team allocation is known to be final.

Recommendation

Key the protected reserve to `totalTeamAllocated - totalTeamReleased` rather than to the constant, so the reserve tracks real obligations. If the conservative constant is deliberate, add an explicit Safe-controlled path to close the team allocation and release the unused remainder. Enforce the funding precondition in the deployment-readiness check described in Q-02.

Status

Resolved

[Q-05] AIMAVesting#createTeamGrant - Team cliff is anchored to token creation, so a late grant is immediately vested

Description

Team vesting is measured from token creation rather than grant creation or acceptance. A team grant created after the 24-month cliff is fully claimable immediately after acceptance.

Vulnerability Details

Affected file: AIMAVesting.sol

```
if (grant.vestingType == AIMADatatypes.VestingType.TEAM) {  
    uint256 teamCliff = uint256(tokenCreationTimestamp) + 24 *  
    uint256(AIMAConstants.MONTH);  
  
    return timestamp >= teamCliff ? grant.amount : 0;  
}
```

Under the documented runbook the anchor is approximately correct: step 4 creates team grants immediately after deployment, so the cliff measured from token creation closely tracks a cliff measured from grant creation. The divergence appears only for grants issued later — replacement hires, or a grant re-approved after cancellation.

Impact

A late grant does not impose a new 24-month lock on its beneficiary. Grant creation is Safe-controlled and this may match intended token-wide vesting policy, so the issue is classified as Q.

Recommendation

Confirm the intended policy. If each grant requires its own cliff, store its acceptance timestamp and anchor vesting to that value. If the token-wide anchor is intended, document that team grants created after month 24 vest immediately.

Status

Resolved

[Q-06] AIMAVesting - Read surface does not mirror the constraints enforced by the write surface

Description

Several views report state that the corresponding state-changing functions would reject. Claimability views ignore the pause condition, and beneficiary enumeration retains cancelled grants. Both mislead integrations in the same way: the view answers a question the transaction will answer differently.

Vulnerability Details

Affected file: `AIMAVesting.sol` — `releasableAmount` does not mirror `claim`'s pause modifier:

```
function releasableAmount(uint256 grantId) public view returns (uint256) {
```

```
function claim(uint256 grantId) external whenNotPaused nonReentrant {
```

Affected file: `AIMAVesting.sol` — `cancelPendingTeamGrant` clears the one-grant mapping and the allocation counters, but leaves the grant ID in the beneficiary's enumeration array:

```
grant.status = AIMADaTypes.GrantStatus.CANCELLED;  
teamGrantIdOf[grant.beneficiary] = 0;  
  
totalTeamAllocated -= grant.amount;  
totalOutstanding -= grant.amount;
```

The contract already implements the correct removal elsewhere: `replacePendingTeamGrantBeneficiary` and `executeRecovery` both use swap-and-pop against `_beneficiaryGrantIds` and `_beneficiaryGrantIndex`. Cancellation simply does not call it.

Related capacity views on the sale can also report remaining amounts after their corresponding lifecycle has ended.

Impact

Off-chain interfaces may show tokens as claimable while claims are paused, causing predictable reverted transactions and misleading user displays. Integrations using `getBeneficiaryGrantIds` must fetch and filter cancelled grants, and enumeration grows with tombstones. No on-chain accounting violation occurs in either case; allocation and claim accounting remain correct.

Recommendation

Make the claimability view mirror the pause and terminal lifecycle conditions, or expose explicit claim-state information alongside the amount. Remove the cancelled ID using the same swap-and-pop bookkeeping already used by beneficiary replacement, or expose a filtered live-grants view.

Status

Resolved

[Q-07] AIMAConstants and AIMAErrors - Unused declarations leave 65% of supply as unenforced allocation metadata

Description

Four allocation constants and four custom errors are declared but never referenced anywhere else in the Solidity repository. The allocation constants may suggest that development, marketing, liquidity, and treasury distributions are enforced on-chain, although AIMAToken only mints the full supply to the Safe.

Vulnerability Details

Affected file: AIMAConstants.sol

```
uint256 internal constant DEVELOPMENT_ALLOCATION = 5_000_000e18;  
uint256 internal constant MARKETING_ALLOCATION = 1_250_000e18;  
uint256 internal constant LIQUIDITY_ALLOCATION = 2_500_000e18;  
uint256 internal constant TREASURY_ALLOCATION = 7_500_000e18;
```

These four constants sum to 16,250,000 AIMA. The documented deployment runbook distributes 5,000,000 to vesting (step 2) and 3,750,000 to the sale (step 3) — 8,750,000 in total. The remaining 16,250,000 AIMA, exactly 65% of total supply, matches the four unused constants exactly and is never moved by any contract, any script, or any documented step. It remains in the Safe under off-chain policy alone.

Affected file: AIMAErrors.sol

```
error InvalidTimestamp();
```

```
error SaleNotConfigured();
```

```
error RecoveryAddressNotSet();
```

```
error InvalidPrivateSaleCloseTimestamp();
```

All events declared in `AIMAEvents.sol` are emitted by at least one production path. The public accounting variables are also ABI-visible and updated at runtime, so they are not classified as unused even where they are not consumed by an internal guard.

Impact

There is no direct runtime vulnerability or bytecode cost from unused internal constants. The declarations increase maintenance noise and may mislead reviewers, token holders, or integrations into assuming that the four named tranches — the clear majority of supply are enforced by the contracts, when no contract references them and no deployment step distributes them.

Recommendation

Remove the unused errors. Either remove the unused allocation constants and document those distributions explicitly as off-chain Safe policy, or enforce them in a dedicated allocation/distribution flow if they are protocol requirements. Preserve public accounting variables unless the project intentionally accepts an ABI-breaking cleanup.

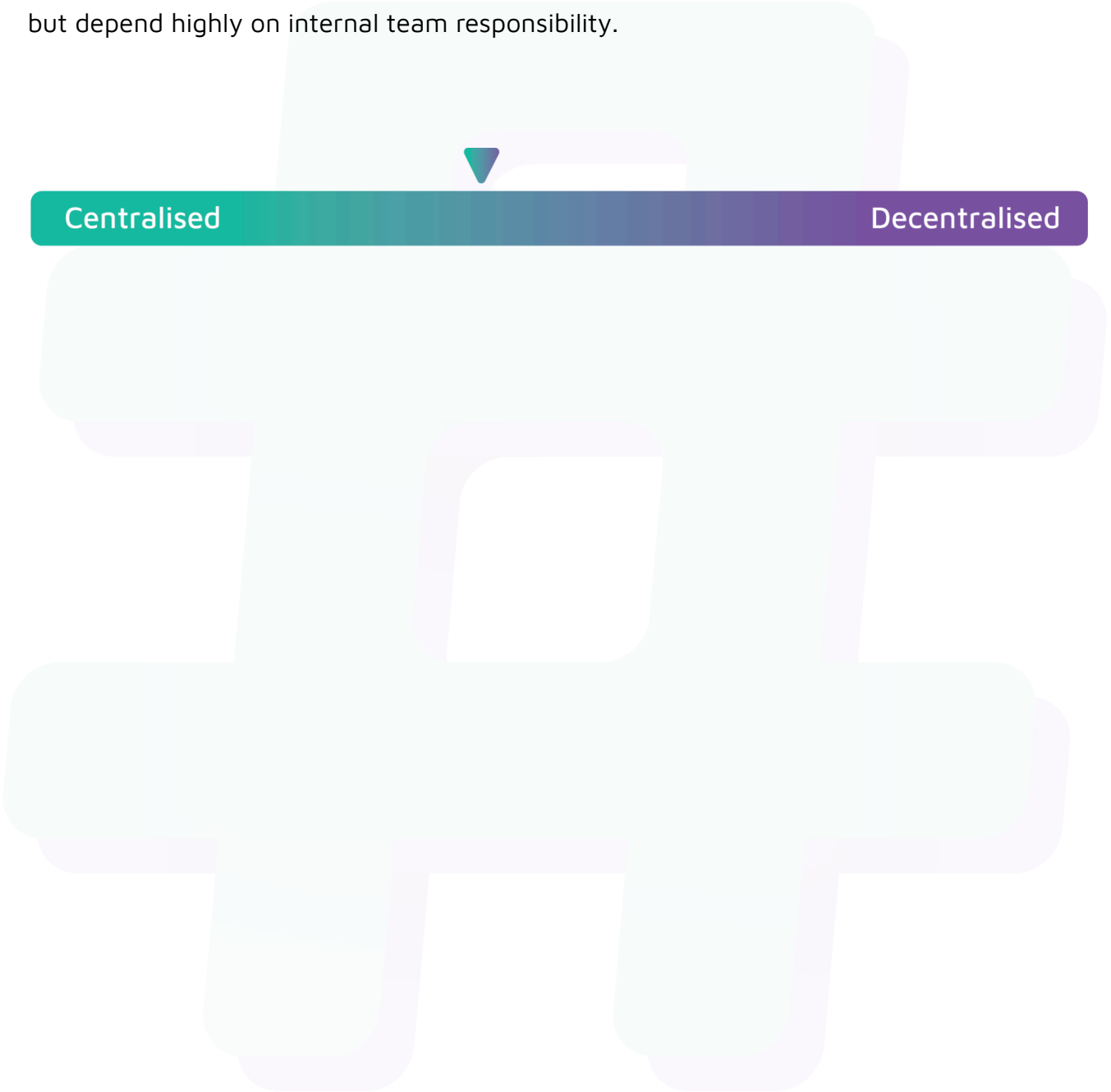
Status

Resolved

Centralisation

The AIMA project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the AIMA project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.